

Архитектурный подход к проектированию клиентской части приложения

- 2.1 Проблемы традиционного подхода к организации кодовой базы во фронтенд-разработке
- 2.2 Методология Feature-Sliced Design: основные понятия и структура слоев.
- 2.3 Ключевые принципы FSD: публичный API, низкая связность (low coupling) и высокое зацепление (high cohesion)
- 2.4 Сравнительный анализ FSD с другими архитектурами фронтенд-приложений

2.1. Проблемы традиционного подхода к организации кодовой базы во фронтенд-разработке

В течение длительного времени стандартным методом организации проектов во фронтенд-разработке являлась декомпозиция кодовой базы по техническому типу. При таком подходе структура каталогов приложения верхнего уровня выглядит следующим образом:

- **components** — содержит все компоненты интерфейса
- **views** — для компонентов целых страниц;
- **store** — для файлов управления глобальным состоянием приложения;
- **api** — для функций отправки сетевых запросов и описания сетевых моделей;
- **utils** — для вспомогательных функций;
- **hooks** — для переиспользуемой реактивной логики.

На начальных этапах жизненного цикла программного продукта техническая декомпозиция обеспечивает быстрый старт разработки за счет простоты и интуитивности структуры. Однако по мере развития приложения, усложнения бизнес-логики и увеличения команды разработчиков традиционный подход порождает ряд серьезных архитектурных проблем, снижающих качество программного обеспечения и увеличивающих стоимость его поддержки.

2.1.1. Техническая размытость и разрастание папок общего назначения

По мере роста проекта папка components неизбежно превращается в плоское неструктурированное хранилище, содержащее сотни файлов. Разработчик сталкивается с ситуацией, когда компонент кнопки находится в той же директории, что и сложный интерактивный модуль. В рамках такой структуры невозможно определить зону ответственности компонента, степень его важности для системы и

безопасность внесения изменений. Попытки структурировать данную папку путем создания вложенных директорий, например, `components/common`, `components/modals`, `components/layout`, носят субъективный характер и со временем приводят к хаосу из-за отсутствия жестких архитектурных правил.

2.1.2. Низкое зацепление кода

При декомпозиции код, отвечающий за реализацию одной конкретной бизнес-фичи, оказывается распределен по всему дереву проекта. Например, для добавления или изменения функционала «Прохождение теста студентом» разработчик вынужден одновременно вносить изменения, например, в следующие файлы:

- `/components/TestPlayer.vue` — для разметки интерфейса;
- `/store/test.ts` — для изменения глобального состояния сессии;
- `/api/test.ts` — для добавления новых запросов к бэкенду;
- `/composables/useTimer.ts` — для логики обратного отсчета.

Такая разрозненность приводит к высокой когнитивной нагрузке и усложнению навигации по проекту.

2.1.3. Сложность удаления функционала

Прямым следствием низкого зацепления является высокая сложность проведения рефакторинга и безопасного удаления неактуального функционала. При изменении требований, в том случае, когда определенная фича более не требуется, ее удаление превращается в сложную задачу. Из-за отсутствия четких границ между модулями, разработчик не может быть уверен, что удаление глобального стора или некоторой утилиты не нарушит работоспособность других страниц. В результате в кодовой базе накапливается мертвый код, который продолжает собираться сборщиком, увеличивая размер итогового бандла приложения и замедляя его загрузку.

2.1.4. Сильная связность и круговые зависимости

Традиционная структура не накладывает никаких ограничений на направление импортов внутри проекта. Любой компонент может импортировать любой другой компонент, стор или утилиту. В условиях отсутствия вертикальной иерархии зависимостей компоненты начинают связываться напрямую. Это приводит к возникновению круговых зависимостей, когда модуль А импортирует модуль В, в свою очередь модуль В зависит от модуля А. Круговые зависимости вызывают трудноуловимые ошибки в рантайме, усложняют процесс сборки приложения и делают модули непереиспользуемыми. Изменение логики отображения элемента на одной странице может каскадно сломать логику на другой.

2.1.5. Отсутствие инкапсуляции и глубокие импорты

При традиционном подходе отсутствует понятие публичного контракта модуля. Компоненты импортируются напрямую. Любое перемещение файлов внутри папки `components` в процессе рефакторинга ломает импорты по всему проекту. Отсутствие «фасадов» делает внутреннюю структуру компонентов открытой для внешнего использования, исключая возможность безопасной внутренней модификации кода.

2.1.6. Конфликты слияния в системе контроля версий

Поскольку все разработчики вынуждены обращаться к папкам общего назначения (добавлять новые файлы в единую папку `components`, регистрировать новые сторы в общем `store`), при совместной работе регулярно возникают конфликты слияния в `Git`. Разрешение таких конфликтов требует дополнительного времени и повышает риски случайного удаления чужого кода, что может поломать работоспособность системы. Кроме того, новый разработчик, приходя в проект, сталкивается с высокой сложностью освоения: ему требуется изучить весь проект целиком, чтобы понять взаимосвязи между разрозненными файлами.

Для разрабатываемой платформы обучения студентов, характеризующейся наличием непересекающихся ролевых интерфейсов (Администратор, Преподаватель, Студент), использование традиционного подхода к разработке клиентской части веб-приложения несет в себе некоторые риски. Без жестких архитектурных правил код личного кабинета студента быстро окажется связанным с интерфейсом администратора через общие папки компонентов и стейта. Это сделает платформу хрупкой, усложнит процесс изолированного тестирования ролевых сценариев и полностью заблокирует возможность легкого последующего расширения функционала платформы.

2.2 Методология Feature-Sliced Design: основные понятия и структура слоев.

Методология Feature-Sliced Design (FSD) представляет собой современный архитектурный подход к проектированию и структурированию кодовой базы клиентских веб-приложений. В отличие от традиционных архитектурных паттернов, разделяющих файлы исключительно по техническому назначению, например, распределение по папкам: `components`, `views`, `stores`, FSD предлагает ориентировать структуру проекта на бизнес-логику и функциональные особенности продукта. Главная цель методологии заключается в снижении связности кода и повышении его сцепленности, что делает проект легко масштабируемым, поддерживаемым и устойчивым к регулярным изменениям требований.

Методология FSD базируется на следующих основных терминах: слои (папки верхнего уровня), слайсы (папки второго уровня) и сегменты (папки третьего уровня) (рис. 1)

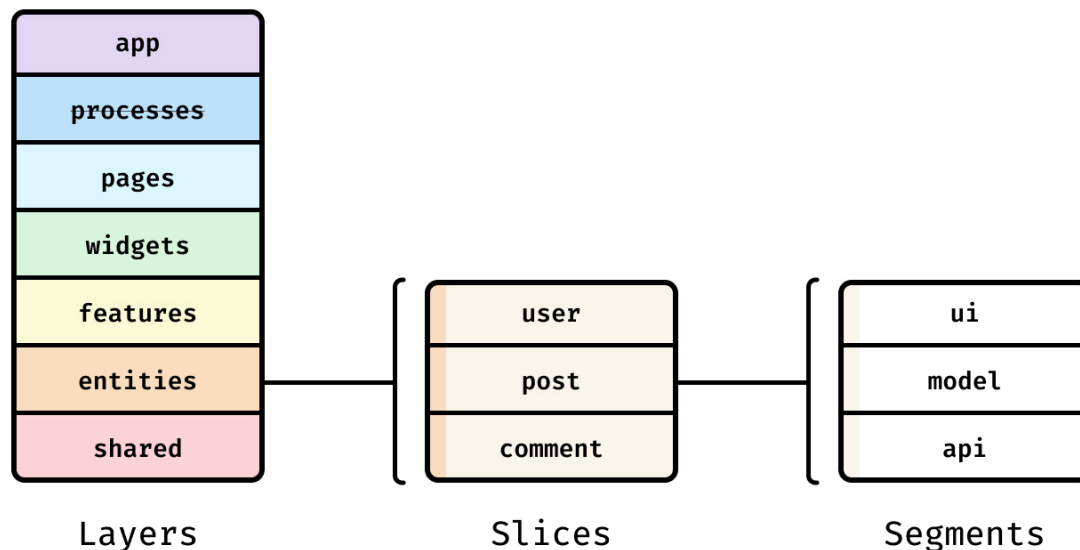


Рис. 1.

2.2.1 Слои

Слои представляют собой верхнеуровневые директории проекта и первый уровень организационной иерархии. В методологии FSD стандартизировано шесть основных слоев (ранее было 7, но слой process признан устаревшим), расположенных в строгой вертикальной иерархии от верхнего (системного) уровня к нижнему (инфраструктурному):

app - слой инициализации приложения. Здесь располагаются всё то, что касается приложения целиком. Этот слой обычно не содержит слайсов, а сразу использует сегменты. Пример того, что обычно располагается на этом слое:

- точка входа в приложение
- подключение глобальных стилей
- конфигурация маршрутизатора
- инициализация корневого стейт-менеджера
- общие провайдеры контекста

pages - слой страниц приложения. Слой содержит компоненты “экранов” (визуальных страниц), которые собираются из нижележащих блоков. Одна страница обычно соответствует одному слайсу, однако, если есть несколько очень похожих страниц, их

можно сгруппировать в один слайс, например, формы регистрации. Страницы на данном слое содержат минимум бизнес-логики и выполняют преимущественно композиционную роль. В слайсе страницы обычно располагается интерфейс страницы, а также состояния загрузки и границы ошибок, также допускается располагать здесь запросы на получение и изменение данных.

widgets - слой самостоятельных блоков интерфейса. Виджеты представляют собой крупные, функционально законченные модули, например, шапка, боковое меню, подвал сайта. Виджеты наиболее полезны, когда они используются на нескольких страницах или когда страница, к которой они принадлежат, имеет несколько больших независимых блоков, и это один из них.

features - слой пользовательских действий. Слой предназначен для основных взаимодействий в приложении, действий, которые важны пользователям. Эти взаимодействия часто затрагивают бизнес-сущности, поскольку сущности — это то, о чём приложение. Слайс фичи может содержать UI для выполнения действия, например, форму, вызовы API, необходимые для выполнения действия, валидацию и внутреннее состояние, фиче-флаги (некоторую конфигурацию).

entities - слой бизнес-сущностей предметной области. Слой содержит бизнес-сущности приложения. Обычно это термины, которые бизнес использует для описания продукта. Например, социальная сеть может работать с бизнес-сущностями, такими как Пользователь, Публикация и Группа. Слайс сущности может содержать хранилище данных, схемы валидации данных, функции запросов API, связанные с сущностями, а также визуальное представление этой сущности в интерфейсе.

shared - слой инфраструктурного кода. Слой формирует фундамент для остальной части приложения. Это место для создания связей с внешним миром, например, бэкенды, сторонние библиотеки, среда выполнения приложения. Также это место для собственных библиотек, сконцентрированных на конкретной задаче. Этот слой, также как и App не содержит слайсов. Слайсы предназначены для разделения слоя на предметные области, но предметные области не существуют в Shared. Это означает, что все файлы в Shared могут ссылаться и импортировать друг друга. Пример того, что может располагаться на этом слое

- API-клиент и функции для выполнения запросов к конкретным эндпоинтам бэкенда
- UI-кит приложения
- коллекция внутренних библиотек

- *переменные окружения, глобальные фиче-флаги и другая глобальная конфигурация для вашего приложения*
- *код, настраивающий систему переводов, а также глобальные строки перевода*

2.2.2 Слайсы

Слайсы — это второй уровень в организационной иерархии FSD. Их основное назначение — группировать код по его значению для продукта, бизнеса или просто приложения. Имена слайсов не стандартизированы, поскольку они напрямую определяются бизнес-областью приложения. Слои Shared и App не содержат слайсов. Это потому, что Shared не должен содержать никакой бизнес-логики, следовательно, не имеет продуктового значения, а App должен содержать только код, касающийся всего приложения, поэтому разделение не требуется

2.2.3 Сегменты

Сегменты — это третий и последний уровень в организационной иерархии, их цель — сгруппировать код по его техническому назначению.

Существует несколько стандартизованных названий сегментов:

- ***ui*** — *все, что связано с отображением UI: UI-компоненты, форматтеры дат, стили и т.д.*
- ***api*** — *взаимодействие с бэкендом: функции запросов, типы данных, мапперы и т.д.*
- ***model*** — *модель данных: схемы, интерфейсы, хранилища и бизнес-логика.*
- ***lib*** — *библиотечный код, необходимый другим модулям в этом слайсе.*
- ***config*** — *конфигурационные файлы и фиче-флаги.*

Также допустимо создавать свои собственные сегменты. Наиболее распространенные места для кастомных сегментов — это слои App и Shared, где слайсы не имеют смысла. При создании своего собственного сегмента необходимо убедиться, что название собственного сегмента описывает, для чего нужно его содержимое, а не чем оно является. Например, components, hooks и types — плохие названия сегментов, потому что они не так полезны при поиске кода.

2.3 Ключевые принципы FSD: публичный API, низкая связанность и высокое зацепление.

Архитектурная методология FSD основана на классических принципах проектирования программного обеспечения, сформулированных в рамках теории

объектно-ориентированного программирования и системного анализа. Ключевыми метриками, определяющими качество архитектуры клиентского приложения, выступают слабая связанность и высокое сцепление, а также строгое соблюдение принципа инкапсуляции через механизмы публичного интерфейса. FSD предлагает инструменты и правила для достижения оптимальных значений этих метрик, что критически важно при разработке расширяемых и масштабируемых приложений.

2.3.1 Принцип публичного API

Публичный API — это контракт между группой модулей, например, слайсом, и кодом, который его использует. Он также действует как ворота, разрешая доступ только к определенным объектам и только через этот публичный API. На практике это обычно реализуется как индексный файл с реэкспортами.

Экспорт из слайса через `index.ts` решает несколько важных задач:

- **Инкапсуляция и сокрытие деталей реализации:** внешние модули не имеют доступа к внутренней структуре слайса. Например, если внутри слайса `features/submit-test` содержатся вспомогательные компоненты разметки, локальные функции валидации и специфические типы, они не экспортируются наружу. Это предотвращает случайное и некорректное использование внутренних утилит слайса в других частях приложения.
- **Безопасность рефакторинга:** разработчик может полностью изменить внутреннюю структуру слайса (переименовать файлы, разбить один крупный компонент на три мелких, переписать логику в сегменте `model`), не опасаясь нарушить работоспособность всего приложения. До тех пор, пока сигнатура экспортов в `index.ts` остается неизменной, внешние зависимости не потребуют корректировки.
- **Декларативность контрактов:** файл `index.ts` служит контрактом модуля, позволяя другому разработчику мгновенно понять, какие компоненты, сторы или методы данный слайс предоставляет для использования на других слоях.

2.3.2. Низкая связанность

Связность (`coupling`) определяет степень взаимозависимости между различными программными модулями. В проектах без строгой архитектуры изменение логики в одном компоненте часто вызывает каскад непредсказуемых ошибок в других частях системы, которые не связан с ним. FSD минимизирует связанность за счет двух фундаментальных ограничений:

- **Вертикальное разделение зависимостей.** Иерархическая структура слоев гарантирует, что изменения на верхних слоях, например, изменение дизайна страницы на слое `pages`, не затронут бизнес-логику на нижних слоях.
- **Запрет на перекрестные импорты.** Слайсы, расположенные на одном и том же слое, не могут импортировать код друг друга. Например, компонент отображения карточки студента не может напрямую импортировать и использовать компонент карточки курса. Если возникает необходимость объединить их функционал, это должно происходить на вышележащем слое.

Благодаря слабой связности модулей в системе достигается высокая изоляция ролевых интерфейсов. Изменения в личном кабинете студента гарантированно не затронут интерфейсы преподавателя или администратора, а обновление конкретной фичи, например, переход на новый плеер видеолекций, потребует изменений только внутри одного изолированного слайса.

2.3.3. Высокое зацепление

Зацепление (*cohesion*) характеризуется степенью нахождения кода, связанного между собой одной целью, в рамках одного модуля. Традиционная структура клиентских веб-приложений, разделяющая файлы по техническому типу (компоненты отдельно, хранилища отдельно, запросы к `api` отдельно), имеет крайне низкое зацепление. Для изменения одной фичи разработчик вынужден вносить правки в файлы, разбросанные по всему проекту.

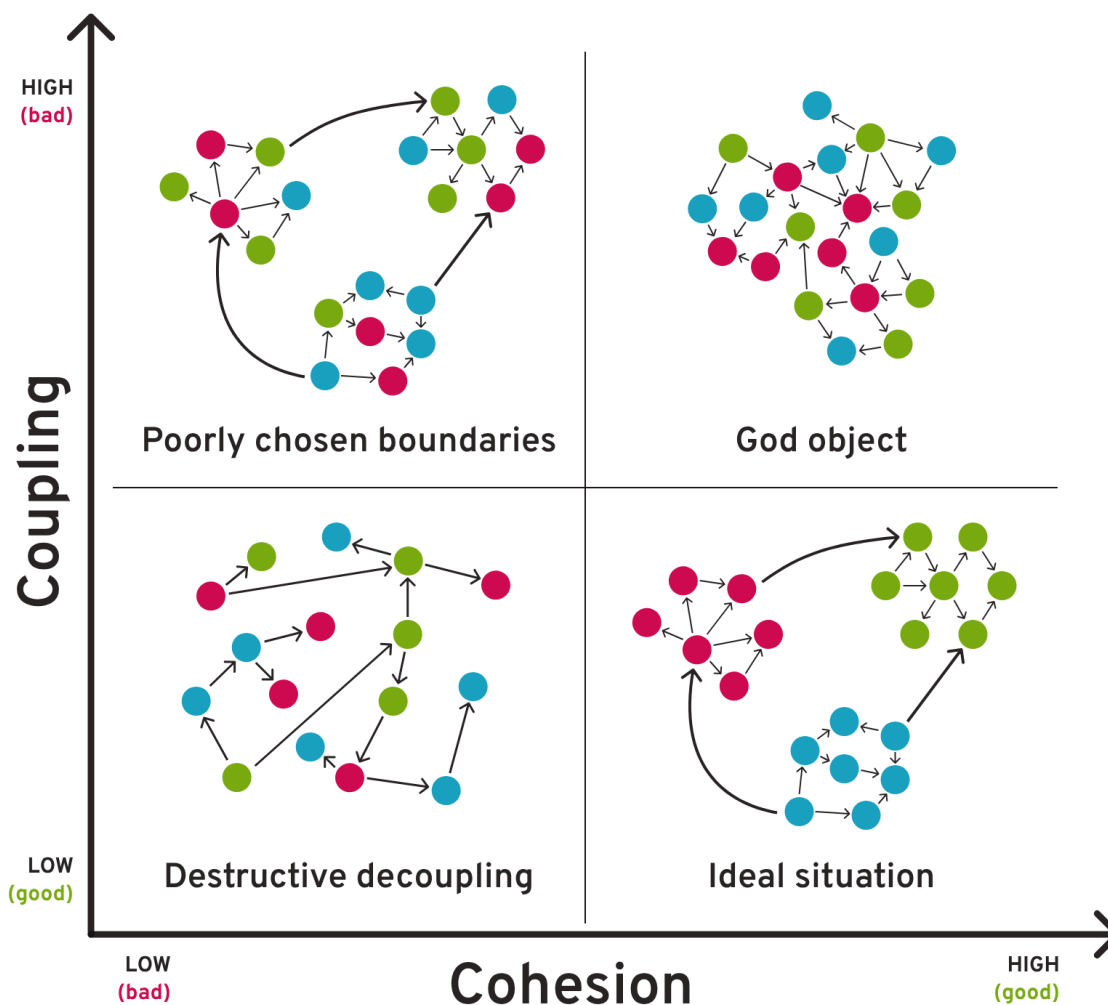
FSD решает эту проблему путем доменно-ориентированного подхода. Все ресурсы, необходимые для функционирования конкретной бизнес-сущности или фичи (UI-компоненты, стили, стейт-менеджеры, API-запросы, типы данных), группируются локально внутри одного слайса и распределяются по сегментам. В связи с этим можно выделить следующие преимущества высокого зацепления в рамках FSD:

- **Локализация контекста.** Разработчик, работающий над модулем, имеет всю необходимую логику, стили и запросы в рамках одной директории. Ему не требуется искать необходимые фрагменты кода по всему проекту.
- **Упрощение совместной разработки.** Поскольку работа над фичами локализована в разных слайсах, минимизируется вероятность возникновения конфликтов слияния в системе контроля версий `Git` при одновременной работе нескольких разработчиков над разными частями платформы.
- **Повышение масштабируемости.** Высокое зацепление гарантирует, что модуль представляет собой «черный ящик», который легко переносить, переиспользовать

или удалять из проекта без риска оставить неиспользуемый («мертвый») код в глобальных папках стилей или стейта.

2.3.4 Вывод

Таким образом, низкая связанность и высокое зацепление (рис. 2)



Взял отсюда -

<https://feature-sliced.design/ru/docs/reference/slices-segments>

при использовании строгих правил публичного API в методологии FSD, создают архитектурный каркас, идеально подходящий для командной разработки и для последующего масштабирования и добавления нового функционала приложения.

2.4 Сравнительный анализ FSD с другими архитектурами фронтенд-приложений

Выбор архитектурного подхода для клиентского веб-приложения всегда представляет собой компромисс между строгостью структуры, скоростью разработки и сложностью поддержки кодовой базы. Для обоснования выбора FSD в рамках разработки платформы обучения студентов необходимо провести сравнительный анализ FSD с альтернативными архитектурными подходами, наиболее распространенными во фронтенд-разработке:

- Классическая (слоистая) архитектура по техническому типу (Layered/Flat Technical). Код группируется по ролям файлов (компоненты, сторы, роуты, утилиты).
- Доменно-модульная архитектура (Domain-Driven Modular): код группируется по бизнес-домена, например, папки /modules/course, /modules/user, но отсутствует строгое вертикальное деление на слои, а связи между модулями не регламентированы.
- Чистая архитектура (Clean Architecture). Перенесенный с бэкенд-разработки паттерн Роберта Мартина, делящий код на концентрические круги (доменные сущности, юзкейсы, адаптеры интерфейсов, внешние фреймворки и UI).

Для удобства сравнения и выявления наиболее сбалансированного решения данные подходы сопоставлены по ключевым критериям качества программного обеспечения (табл. 3).

Таблица 3.

Сравнительный анализ архитектурных подходов для фронтенд-приложений

Критерий сравнения	Layered/Flat Technical	Domain Modular	Clean Architecture	Feature-Sliced Design
Слабая связанность	Очень низкая (хаотичные связи)	Средняя (риск cross-imports)	Высокая	Высокая (регулируется слоями)
Зацепление кода	Низкое	Высокое	Высокое	Высокое (через слайсы и сегменты)
Масштабируемость кода	Низкая	Средняя	Высокая	Высокая
Порог входа	Минимальный	Средний	Очень высокий	Средний
Объем шаблонного кода	Минимальный	Средний	Избыточно высокий	Умеренный
Применимость для UI-фреймворков	Высокая	Высокая	Низкая (требует адаптеров)	Высокая (UI-first подход)

Простота расширения	Низкая	Средняя	Высокая	Высокая
----------------------------	--------	---------	---------	---------

Сравнение FSD с альтернативными подходами позволяет выявить ключевые преимущества и недостатки каждого подхода в контексте решаемой задачи.

Традиционная структура привлекает разработчиков отсутствием шаблонного кода и высокой скоростью создания базового прототипа. Однако по мере добавления новых требований в продукте приложение в плоской структуре быстро превращается в монолитный узел неявных зависимостей. FSD же требует чуть больше времени на начальную настройку слоев и сегментов, но взамен гарантирует стабильную скорость разработки и безопасность рефакторинга на протяжении всего жизненного цикла продукта.

Доменно-модульный подход решает проблему низкого зацепления, группируя все файлы, связанные одним логическим назначением в одну директорию. Главный недостаток этого подхода — отсутствие вертикальной иерархии. Без правил, запрещающих перекрестные импорты, модули неизбежно начинают зависеть друг от друга напрямую. Это приводит к циклическим зависимостям. FSD устраняет эту проблему за счет введения вертикальных слоев, строго определяющих направление потока зависимостей только сверху вниз.

Чистая архитектура предлагает максимальную независимость бизнес-логики от внешних фреймворков и UI. Однако при переносе этого паттерна на фронтенд возникает избыточное усложнение. Современные клиентские приложения, разработанные на фреймворках реактивны по своей природе и тесно связаны с DOM. Попытка изолировать доменные сущности от реактивного состояния, создавать адаптеры данных и выделять отдельные слои репозитория для простых CRUD-запросов приводит к лавинообразному росту шаблонного кода. В отличие от Clean Architecture, методология FSD ориентирована непосредственно на клиентскую веб-разработку. Она предлагает декомпозицию, где реактивный стейт-менеджер и API-клиент органично интегрируются в сегменты `model` и `api` каждого слайса, не требуя создания лишних слоев абстракции.

Таким образом, сравнительный анализ показывает, что FSD представляет собой оптимальный баланс между модульной строгостью, скоростью разработки и простотой поддержки. Для платформы обучения студентов выбор FSD обеспечивает масштабируемость и дает гарантию того, что новые функциональные модули, создаваемые в будущем, могут быть интегрированы в систему в качестве изолированных слайсов без риска разрушить существующие связи в приложении.